



Complemento Práctico: El Arte de la Adaptabilidad

Llegamos a la cúspide de la Programación Orientada a Objetos. Si la Herencia es la estructura (el esqueleto), el **Polimorfismo** es el movimiento. Es la capacidad de que un mismo mensaje ("moverse", "imprimirse") sea interpretado de formas distintas según quién lo reciba.

En este cierre de unidad, analizaremos la mecánica de la **Sobrescritura (@Override)**, aprenderemos a extender lógica usando **super** en lugar de borrarla, y conoceremos la herramienta opuesta al polimorfismo: la palabra clave **final**, que sirve para poner límites inquebrantables en nuestro código.

Contenido

Polimorfismo y Enlace Dinámico

Cómo un mismo mensaje genera múltiples respuestas según el objeto receptor.

Sobrescritura de Métodos (@Override)

Mecanismo para redefinir comportamientos heredados de forma segura.

Uso de super

Técnica para extender funcionalidad del padre sin reemplazarla completamente.

Principio Open/Closed

Diseño polimórfico vs. uso incorrecto de instanceof.

Palabra clave final

Establecer límites inquebrantables en variables, métodos y clases.

Un Mensaje, Múltiples Respuestas



El ejemplo por excelencia es `toString()`. Java nos permite tratar a todos los objetos como `Object`. Pero cuando llamamos al método, ocurre la magia del **Enlace Dinámico**.

Java no decide qué método ejecutar al **compilar** (que ve solo un `Object`), sino al **ejecutar** (que ve la `Persona` real en memoria). Este fenómeno es el corazón del polimorfismo.

El compilador ve un tipo, pero la máquina virtual ejecuta según el objeto real en memoria.

Sin Sobrescritura

Se ejecuta la versión del padre.
Resultado: Persona@15db9742

Con Sobrescritura

La versión del hijo "tapa" a la del padre. Resultado: Soy Ana

```
Object obj = new Persona("Ana");
// El compilador ve un "Object"
// Pero en ejecución, Java busca en "Persona"
System.out.println(obj.toString());
```

El Cinturón de Seguridad del Programador

La anotación `@Override` es opcional para que el código funcione, pero **moralmente obligatoria** para que el código sea seguro. ¿Por qué? Porque nos protege de los "dedos torpes" (errores de tecleo) y de cambios futuros en la clase padre.

El Error Silencioso

Sin `@Override`, un error de tecleo como `ladar()` en lugar de `ladrar()` crea un método NUEVO y distinto. Java cree que es intencional.

Consecuencia: Cuando llames a `animal.ladar()`, se ejecutará el método genérico del padre. Tu perro quedará mudo y no sabrás por qué.

La Protección Activa

Con `@Override`, el compilador verifica que realmente estás sobrescribiendo un método existente. Si hay un error de tecleo, obtienes un **error de compilación** inmediato.

Lección: `@Override` le dice al compilador: "Verifica por mí que realmente estoy sobrescribiendo un método de la clase padre".

```
public class Perro extends Animal {
    @Override
    public void ladar() { ... }
    // ¡ERROR DE COMPILACIÓN!
    // "No existe 'ladar' en Animal"
}
```

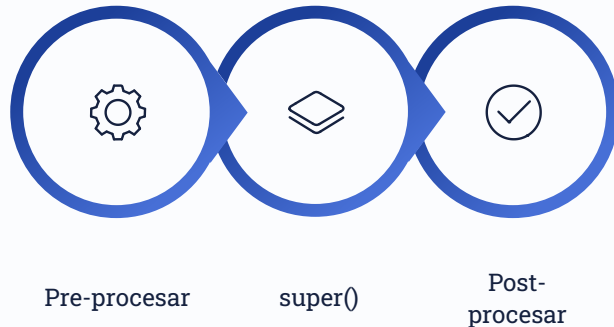
No Reemplazar, sino Aumentar

A veces no queremos borrar lo que hacía el padre, sino **envolverlo** o agregarle funcionalidad extra. Aquí es donde `super` se convierte en nuestra herramienta más poderosa.

El patrón "Sándwich" (también conocido como Decorator) nos permite:

1. Hacer algo antes (Pre-procesamiento)
2. Llamar a la lógica original del padre con `super.metodo()`
3. Hacer algo después (Post-procesamiento)

Esta técnica es fundamental cuando queremos **extender** comportamiento sin duplicar código. Es la esencia de la reutilización inteligente.



Este patrón garantiza que la lógica compleja del padre se mantiene intacta mientras agregamos nuestra especialización.

Ejemplo Práctico: Cálculo de Salario

Un empleado cobra salario básico (Padre), pero el Gerente cobra eso MÁS un bono de \$5000.

```
@Override
public double calcularSueldo() {
    // 1. Reutilizo el cálculo del padre
    double basico = super.calcularSueldo();
    // 2. Agrego mi especialización
    return basico + 5000;
}
```

Falso Polimorfismo y Principio Open/Closed

Si te encuentras escribiendo cadenas infinitas de `if (x instanceof A)`, **estás programando mal**. Estás violando el Principio Open/Closed (Abierto a extensión, cerrado a modificación). Este es uno de los anti-patrones más comunes que encontrarás en código legacy.

El problema fundamental es que cada vez que agregues un nuevo tipo (por ejemplo, una Vaca), tendrás que volver a este código y modificarlo. Esto crea un punto de fallo centralizado y viola el principio de que el código debe estar cerrado a modificación pero abierto a extensión.

✗ Mal Diseño: Manual y Rígido

```
if (animal instanceof Perro) {
    ((Perro)animal).ladrar();
} else if (animal instanceof Gato) {
    ((Gato)animal).maullar();
}
// Si agrego Vaca, debo editar AQUÍ
```

Este código es frágil, difícil de mantener y viola principios SOLID.

✓ Buen Diseño: Polimórfico y Flexible

```
// Método común en padre Animal
animal.hacerRuido();

// Cada clase sabe qué hacer
// No preguntamos el tipo
```

Si mañana agrego una Vaca, este código **no se toca**. La Vaca sabrá hacer "Muu" por sí misma.

Los 4 Usos de Final

La palabra clave `final` es la herramienta para decir "Esto no se cambia más". Es el candado de Java, y tiene cuatro aplicaciones distintas que debes dominar. Cada una proporciona un tipo diferente de inmutabilidad y garantías de seguridad.

01

En una Variable: Constante

```
final double PI = 3.14;
```

Si intentas `PI = 4;`, el compilador da error.

02

En un Atributo: Inmutabilidad

```
private final String DNI;
```

Debe inicializarse en el constructor. Una vez asignado, es intocable.

03

En un Método: No Sobrescribible

```
public final void respirar()
```

Los hijos heredan pero no pueden modificar. Útil para algoritmos de seguridad.

04

En una Clase: No Heredable

```
public final class String
```

Nadie puede crear subclases. Es el fin de la línea genealógica.

Final en Variables y Atributos

Constantes del Sistema

Cuando declaras una variable como `final`, estás creando una **constante**. Es una convención en Java escribir estas constantes en MAYÚSCULAS con guiones bajos.

```
public final double PI = 3.14159;  
public final int MAX_INTENTOS = 3;  
public final String NOMBRE_APP = "MiSistema";
```

Estas constantes son útiles para valores que nunca deben cambiar durante la ejecución del programa. Si alguien intenta modificarlas, el compilador lo impedirá.

Atributos Inmutables

En el caso de atributos de instancia, `final` garantiza que cada objeto tendrá un valor que se asigna una sola vez (típicamente en el constructor) y nunca cambia.

```
public class Persona {  
    private final String DNI;  
  
    public Persona(String dni) {  
        this.DNI = dni; // Única asignación  
    }  
}
```

Final en Métodos y Clases

Los usos de `final` en métodos y clases son herramientas de **diseño arquitectónico**. Te permiten establecer límites claros sobre qué partes de tu código pueden ser extendidas o modificadas por otros desarrolladores (o por ti mismo en el futuro).

Final en Métodos

Un método `final` no puede ser sobrescrito por las clases hijas. Los hijos lo heredan y pueden usarlo, pero no pueden cambiar su implementación.

```
public class Animal {  
    public final void respirar() {  
        // Algoritmo crítico  
        // Nadie puede modificarlo  
    }  
}
```

¿Cuándo usar? Para algoritmos de seguridad, validaciones críticas, o comportamientos que DEBEN ser consistentes en toda la jerarquía.

Final en Clases

Una clase `final` no puede ser heredada. Es el fin absoluto de una línea de herencia. Nadie puede crear subclases de ella.

```
public final class String {  
    // Implementación intocable  
}  
  
// Esto daría ERROR:  
// class MiString extends String
```

¿Cuándo usar? Para clases de utilidad críticas (como `String`, `Math`) donde la extensión podría comprometer la seguridad o consistencia del sistema.

"Usa final cuando necesites garantías inquebrantables. Es mejor ser explícito sobre las restricciones que descubrir problemas en producción."

Fin del Módulo de POO Avanzada

Hemos completado el núcleo duro de Java. Este viaje nos ha llevado desde los conceptos básicos hasta las técnicas más avanzadas de la Programación Orientada a Objetos. Ahora posees las herramientas fundamentales que todo desarrollador profesional debe dominar.



Herencia

Para reutilizar estructura mediante extends y construir jerarquías lógicas.



Encapsulamiento

Manejo de protected y super para controlar el acceso heredado.



Abstracción

Definición de contratos mediante clases y métodos abstract.



Polimorfismo

Flexibilidad de ejecución mediante Sobrescritura y Enlace Dinámico.

La Regla de Oro del Diseño

"Programa para una interfaz (padre), no para una implementación (hijo)"

Esta máxima resume todo lo aprendido. Cuando diseñes tus sistemas, piensa en abstracciones y contratos, no en detalles concretos. Así lograrás código flexible, mantenible y profesional.

Aplica estos conceptos con confianza, porque ahora comprendes no solo el *cómo*, sino también el *por qué* detrás de cada decisión de diseño.